

PROYECTO DE CURSO

Documento de Apoyo 02: (Semana 6)

750021C - DESARROLLO DE SOFTWARE II



FACULTAD DE INGENIERÍA
Escuela de Ingeniería de Sistemas y
Computación

FEBRERO - JUNIO / 2026

Contenerización de la Aplicación UniTicket

1. Objetivos

- 1.1. Estandarizar el entorno de ejecución de la aplicación mediante el uso de contenedores.
- 1.2. Configurar la persistencia de datos y la comunicación entre servicios.
- 1.3. Asegurar que el proyecto sea ejecutable por Jenkins en el VPS de forma automatizada.

2. Información Clave

Se debe garantizar que el software funcione igual en el computador del estudiante que en el servidor de la universidad o cualquier otro host. Para ello utilizaremos:

- **Dockerfile:** El archivo de configuración para construir la imagen de la aplicación.
- **Docker Compose:** La herramienta para orquestar la App y el motor de Base de Datos.
- **Volúmenes:** El mecanismo para que los datos de los tickets no se pierdan al reiniciar los contenedores.
- **Redes de Docker:** Aislamiento para que los contenedores del Grupo A no interfieran con los del Grupo B.

3. Procedimientos

A. Creación del Dockerfile

Dependiendo del Gestor de Contenedores a usar y del lenguaje de programación elegido para el desarrollo de la aplicación, se debe crear el archivo de configuración de contenedores. A partir de este momento, los ejemplos se basarán en la elección de Docker y Python.

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

B. Configuración de Orquestación

Crear el archivo para levantar ambos servicios.

Importante: Cada grupo debe usar su puerto asignado (ej. 8081).

docker-compose.yml

```
name: uniticket-grupo-X
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: uniticket_db
      POSTGRES_PASSWORD: password_seguro
    volumes:
      - db_data:/var/lib/postgresql/data

  web:
    build: .
    ports:
      - "700X:5000"
    depends_on:
      - db
    environment:
      DATABASE_URL:
postgres://postgres:password_seguro@db:5432/uniticket_db

volumes:
  db_data:
```

C. Prueba de Ejecución Local

Antes de subir a GitHub, el estudiante debe validar en su terminal:

1. `docker-compose build` (Construir la imagen).
2. `docker-compose up -d` (Levantar servicios en segundo plano).
3. Verificar acceso en el navegador.

4. Resultados Esperados

- La aplicación UniTicket debe ser capaz de conectarse a la base de datos dentro de un entorno de contenedores.
- El código subido a la rama `develop` debe incluir los archivos de Docker configurados y funcionales.
- La base de datos debe persistir la información aunque el contenedor se detenga.

5. Solución de Problemas

Problema	Posible Causa	Solución Sugerida
Error: Port already in use	Otro proceso (o el Jenkins) está usando el puerto asignado.	Revisar el <code>docker-compose.yml</code> y asegurar que el puerto de la izquierda sea el asignado al grupo (ej. 7001).
La App no conecta a la DB	El servicio web intenta conectar a <code>localhost</code> en lugar de al nombre del servicio <code>db</code> .	Cambiar el host de conexión en el código de la app por el nombre definido en el <code>docker-compose.yml</code> ("db" en el ejemplo).
Cambios en el código no se ven	El contenedor está corriendo una versión vieja de la imagen.	Ejecutar <code>docker-compose up --build</code> para forzar la reconstrucción.